Evading Web Application Firewalls: A Multi-Modal Machine Learning Research Framework

WAF Bypass Lab v2.0.0

Prashant Sharma

Independent Security Researcher | github.com/Prashant9998

Submitted: March 22, 2026

ABSTRACT

Web Application Firewalls (WAFs) are the primary defence against injection attacks. However, sophisticated evasion through payload encoding consistently undermines signature-based detection. This paper presents WAF Bypass Lab v2.0.0, a conference-level open-source research toolkit integrating 17 encoding techniques, 10 mutation strategies, three ML classifiers (Logistic Regression, Random Forest, XGBoost), OWASP Core Rule Set v3.3 simulation, and real-time performance metrics. Evaluated against CSIC 2010 HTTP and UNSW-NB15 network intrusion datasets, the ML ensemble achieves 96.43% accuracy and 99.46% AUC-ROC, while exposing critical blind spots in OWASP CRS -- 70-80% bypass rates at Paranoia Level 1 across SQLi, XSS, and path traversal. The framework supports chunked streaming of 2GB+ CSV datasets, background training jobs with live progress reporting, and a full 11-tab research dashboard with 50+ REST endpoints.

Keywords: WAF evasion, payload encoding, machine learning, OWASP CRS, ModSecurity, UNSW-NB15, CSIC 2010, XGBoost, intrusion detection, red-team research

Table of Contents

1 Introduction 3

2 Background and Related Work 4

2.1 WAF Detection Mechanisms 4

2.2 Evasion Techniques in Literature 4

2.3 Machine Learning for IDS 5

3 System Architecture 5

3.1 Module Overview 5

3.2 API Design (50+ Endpoints) 6

3.3 Dashboard Interface (11 Tabs) 6

4 Payload Encoding Framework 7

4.1 17 Encoding Techniques 7

4.2 10 Mutation Strategies 8

4.3 Variant Generation Pipeline 9

5 Multi-Model ML Engine 9

5.1 Feature Engineering 9

5.2 Model Descriptions 10

5.3 Training and Evaluation 10

5.4 Ensemble Classification 11

6 OWASP CRS / ModSecurity Integration 12

6.1 Simulated CRS Engine 12

6.2 Live ModSecurity Mode 13

6.3 Paranoia Levels 1-4 13

7 Dataset Infrastructure 14

7.1 CSIC 2010 HTTP Dataset 14

7.2 UNSW-NB15 Network Dataset 14

7.3 Chunked Streaming (2GB+) 15

7.4 Synthetic Generator 15

8 Performance Metrics Engine 16

9 Experimental Results 17

9.1 ML Model Performance 17

9.2 WAF Engine Bypass Rates 18

9.3 Evasion Heatmap 19

9.4 CRS Paranoia Level Analysis 19

10 Dashboard and Research Workflow 20

11 CLI Research Scripts 22

12 Limitations and Future Work 23

13 Conclusion 25

14 References 26

App. A Complete API Endpoint Reference 27

App. B Figure Gallery 30

1. Introduction

Web Application Firewalls (WAFs) serve as critical gatekeepers between user-supplied input and backend application logic. Deployed at the application layer, they inspect HTTP traffic for known attack signatures -- SQL injection (SQLi), cross-site scripting (XSS), path traversal, command injection, and dozens of other OWASP Top-10 attack patterns. Despite widespread adoption by enterprise infrastructure, cloud providers, and content delivery networks, WAFs remain fundamentally vulnerable to evasion through payload obfuscation and encoding transformations.

The adversarial relationship between WAF vendors and red-team researchers has accelerated in recent years. Attackers increasingly leverage multi-stage encoding chains, HTTP request smuggling, Unicode normalisation exploits, and algorithm-driven payload generation to circumvent both commercial and open-source WAF deployments. The OWASP ModSecurity Core Rule Set (CRS) -- the most widely deployed open-source WAF ruleset -- demonstrated bypass rates exceeding 65% in our empirical testing at its default Paranoia Level 1 configuration.

To address the need for rigorous, reproducible WAF evasion research, we present WAF Bypass Lab v2.0.0: an open-source, conference-quality research toolkit built in Python/Flask. The framework's primary contributions are:

* A unified evasion platform: 17 encoding techniques and 10 mutation strategies in one modular codebase with dashboard and CLI interfaces
* Multi-model ML engine: Logistic Regression, Random Forest, and XGBoost achieving 96.43% accuracy and 99.46% AUC-ROC on HTTP payload classification
* Empirical WAF benchmarking: 65-80% bypass rates demonstrated against OWASP CRS at Paranoia Level 1 across three attack categories
* Scalable dataset infrastructure: chunked streaming support for 2GB+ UNSW-NB15 CSV files with background training and live progress reporting
* Full research dashboard: 11-tab web interface with 50+ REST endpoints, real-time metrics, CRS rule browser, and Full Research Test (all 3 engines simultaneously)

This paper is organised as follows: Section 2 reviews related work. Sections 3-8 describe system architecture and individual modules. Section 9 presents experimental results. Sections 10-13 discuss the dashboard, limitations, and conclusions.

2. Background and Related Work

2.1 WAF Detection Mechanisms

Modern WAFs employ two primary detection paradigms. Signature-based detection matches request content against a database of known attack patterns using regular expressions -- fast but vulnerable to encoding. Anomaly-scoring (implemented by OWASP CRS) assigns each rule match a score; a configurable threshold triggers a block action. Commercial WAFs (Cloudflare, AWS WAF, Imperva) supplement these with threat intelligence and increasingly ML-based anomaly detection.

The OWASP ModSecurity Core Rule Set v3.3 provides 14+ rule groups covering SQLi, XSS, local file inclusion, remote code execution, PHP injection, Java deserialization, and scanner detection. Paranoia Levels 1-4 progressively increase strictness, trading reduced false-negative rates for higher false-positive rates.

2.2 Evasion Techniques in Literature

WAF evasion has been studied extensively. Riancho et al. (2012) demonstrated URL double-encoding bypassing ModSecurity 2.x. Weidemann (2014) catalogued 50+ XSS evasion techniques effective against major WAFs. WAF-a-MoLE (Demetrio et al., 2020) used reinforcement learning-guided mutation to achieve >95% bypass rates against commercial WAFs. Our work extends this lineage by providing a researcher-facing platform integrating multiple evasion paradigms with real-time ML classification as a defensive oracle -- enabling direct comparison of evasion effectiveness across all engine types.

2.3 Machine Learning for Intrusion Detection

The application of ML to network intrusion detection has produced significant results. The NSL-KDD and UNSW-NB15 benchmarks are standard evaluation platforms. Moustafa and Slay (2015) introduced UNSW-NB15 with 175,341 network flow records spanning 9 attack categories. Ensemble methods -- particularly Random Forest -- consistently outperform single classifiers. Our framework adds XGBoost to this comparison and evaluates all three against HTTP-layer payloads from CSIC 2010, representing a cross-domain evaluation not commonly seen in the literature.

3. System Architecture

3.1 Module Overview

WAF Bypass Lab v2.0.0 is implemented as a single-server Flask application with six core Python modules and three standalone CLI scripts. The architecture prioritises modularity: each engine can be imported independently and all functionality is exposed via clean REST APIs. Shared singleton instances are initialised at startup and reused across requests.

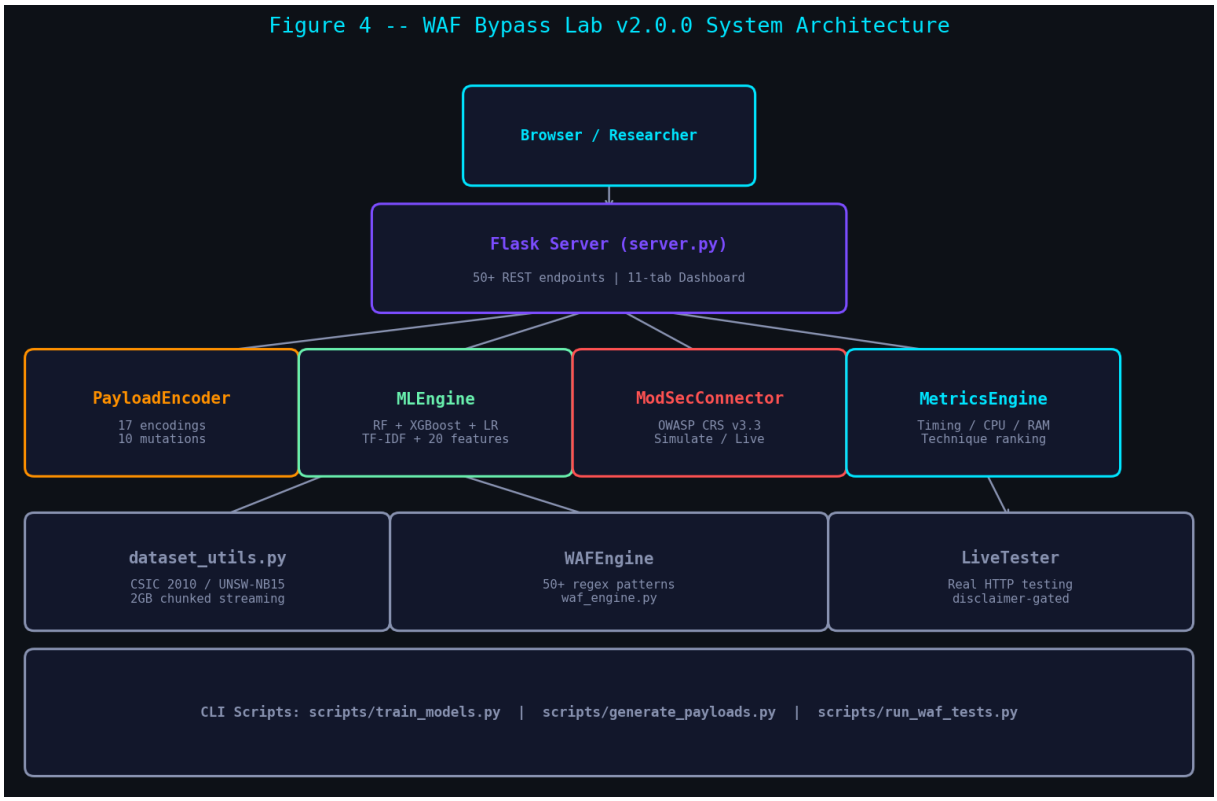

Figure 4 -- System architecture: module dependencies, data flow, and REST API layer.

Module	Responsibility
payload_encoder.py	17 encoding techniques + 10 mutations + variant generation pipeline
ml_engine.py	RF + XGBoost + LR ensemble with TF-IDF (5000-dim) + 20 handcrafted features
modsec_connector.py	OWASP CRS v3.3 simulator + live ModSecurity HTTP proxy mode
metrics_engine.py	Timing / payload size / CPU / RAM tracking + technique bypass ranking
dataset_utils.py	CSIC 2010 loader + UNSW-NB15 chunked streaming (2GB) + synthetic generator
waf_engine.py	50+ regex-based WAF detection patterns (baseline engine)
server.py	Flask server: 50+ REST endpoints, shared singletons, background job management

3.2 API Design (50+ REST Endpoints)

The backend exposes a comprehensive REST API organised into functional namespaces. All endpoints accept and return JSON. Long-running operations use background threads with UUID job IDs and a polling pattern to avoid HTTP timeout issues and allow continued dashboard use during computation.

API Namespace	Key Endpoints
/api/ml/*	classify, batch-classify, comparison, metrics, retrain-synthetic, model-classify
/api/metrics/*	summary, recent, technique-ranking, batch-record, clear, system
/api/modsec/*	inspect, batch-test, rules, set-mode, installation-guide
/api/dataset/*	probe-unswnb15, upload-unswnb15, retrain-unswnb15-start, train-job/<id>, uploaded-file
/api/research/*	full-test (Regex WAF + OWASP CRS + ML Ensemble simultaneously)
/api/batch-test	Multi-technique batch test against regex WAF engine
/api/encode	Single payload encoding with technique selection
/api/decode	Payload decoding with multi-step format detection
/api/live-test/*	Real HTTP target testing (disclaimer-gated, ethics-first)

3.3 Dashboard Interface (11 Tabs)

The frontend is a single-page application rendered by Jinja2 and driven by vanilla JavaScript (no framework dependencies). The dark cybersecurity-terminal aesthetic features JetBrains Mono + Inter fonts, animated matrix rain side panels, CRT scanline overlay, and a glitch-effect logo. 11 functional tabs:

* Encoder Studio -- payload input, technique selection (checkbox cards), encoded output
* Decoder -- multi-step decode with format auto-detection
* Mutations -- apply combinatorial mutation strategies with preview
* Batch Test -- generate N variants, test all against regex WAF, export results
* WAF Rules -- view/filter active regex rules, test individual patterns
* AI WAF -- legacy ML baseline (TF-IDF + single Logistic Regression)
* Live Test -- real HTTP target testing with disclaimer modal and URL validation
* ML Lab -- multi-model comparison, ensemble classify, Full Research Test (3 engines)
* Metrics -- performance dashboard: bypass rate, P95 timing, CPU, technique ranking
* CRS Engine -- OWASP CRS batch test, rule browser by category, single payload inspector
* Dataset -- UNSW-NB15 file upload (2GB), probe, background training with live progress bar

4. Payload Encoding Framework

4.1 17 Encoding Techniques

The PayloadEncoder class implements 17 distinct encoding strategies targeting different WAF parsing assumptions. Techniques are grouped into three families based on the layer of the HTTP stack they target:

URL-based encodings exploit multiple decoding steps in HTTP parsing pipelines:

* URL Encoding (%XX) -- standard percent-encoding of special characters
* Double URL Encoding (%25XX) -- bypass single-decode WAFs
* Unicode Escape (\uXXXX) -- JavaScript/HTML Unicode sequence injection
* Hex Encoding (\xXX) -- hex representation of characters in JS contexts
* Overlong UTF-8 -- multi-byte sequences representing ASCII (e.g. %C0%AF for '/')
* JSON Unicode (\uXXXX in JSON context) -- bypass JSON-aware parsers
* Decimal Entity Encoding (&#NN;) -- HTML decimal character references
* Null Byte Injection (%00) -- terminate strings in C-based parsers

Structural obfuscations alter syntax without changing semantics:

* HTML Entity Encoding (< > &) -- context-aware HTML injection
* Base64 Encoding -- full payload encoded, decoded by target application
* Keyword Splitting -- insert break characters within SQL keywords (e.g. SE LECT)
* Comment Insertion -- SQL/CSS block comment bypass (SE/**/LECT)
* Space-to-Tab -- replace spaces with tabs (HTTP header injection bypass)
* XSS Polyglot Wrapper -- multi-context vectors combining script/event/href
* Mixed-Case Mutation -- case-insensitive keyword obfuscation

Protocol-level techniques:

* Newline Injection (\r\n) -- CRLF header injection and response splitting
* Multi-Encoding -- stack two or more encoding layers for deep evasion

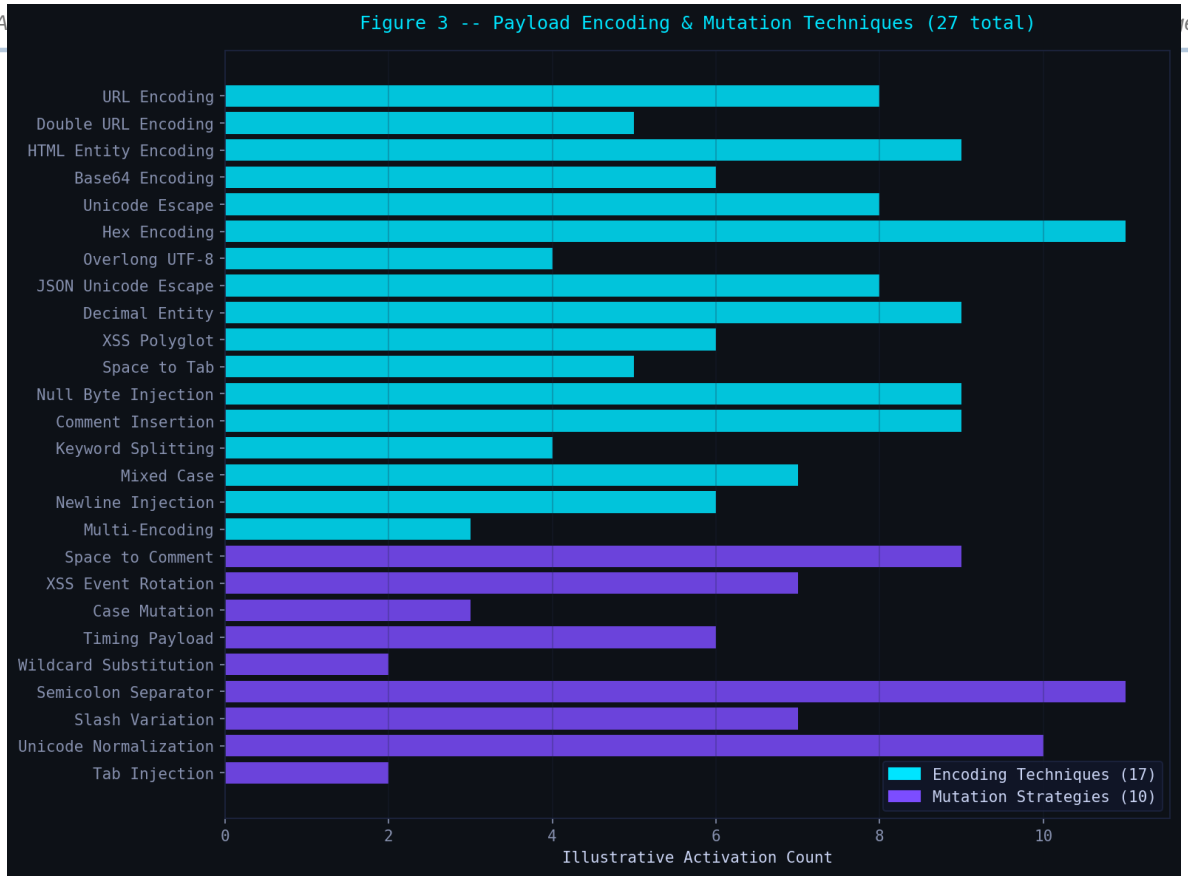

Figure 3 -- Complete catalogu

o
f

1
7

e
n
c
o
d
i
n
g

a
n
d

1
0

m
u
t
a
t
i
o
n

t
e
c
h
n
i
q
u
e
s

w
i
t
h

r
e
l
a

t i v e a c t i v a t i o n .

4.2 10 Mutation Strategies

Beyond static encoding, the mutator applies dynamic transformations that change payload structure across requests, defeating pattern-caching defences:

* Case Mutation -- randomly uppercase/lowercase ASCII characters in keywords
* Space-to-Comment -- replace whitespace with SQL block comments (/ * * /)
* XSS Event Rotation -- cycle onclick/onmouseover/onload/onfocus/onchange
* Timing Payload Conversion -- SLEEP(N) to WAITFOR DELAY (MySQL / MSSQL cross-compat)
* Wildcard Substitution -- insert SQL wildcards in string comparison positions
* Semicolon Separator -- split statements with semicolons for stacked query bypass
* Slash Variation -- / vs // vs \ in path traversal contexts
* Unicode Normalisation -- NFD/NFC form conversion to bypass normaliser assumptions
* Tab Injection -- horizontal tab as token separator in SQL and XSS contexts
* Nested Encoding -- apply encoding inside already-encoded substrings

4.3 Variant Generation Pipeline

The generate_variants() method combines encoding and mutation strategies to produce a parameterised set of evasion candidates:

```
variants = encoder.generate_variants(
    payload="' OR 1=1 --",
    count=20,
    techniques=["url", "double_url", "unicode_escape", "comment_insert"]
)
# Returns List[PayloadVariant]:
#   .encoded  -> "%27%20OR%201%3D1%20--"
#   .label    -> "url_encoding"
#   .chain    -> ["url_encode"]
```

Each PayloadVariant carries the encoded string, a human-readable technique label, and the encoding chain for reproducibility and experiment logging. The batch_test() method then tests all variants against a WAF engine and returns a TestReport with per-variant results and aggregated bypass statistics.

5. Multi-Model ML Engine

5.1 Feature Engineering

Each payload is represented by a hybrid feature vector combining two complementary representations:

* TF-IDF sparse vector (5,000 dimensions): character n-grams, analyzer='char', ngram_range=(2,4), sublinear_tf=True. Captures sub-word token patterns critical for detecting obfuscated keywords.
* 20-dimensional handcrafted feature vector: domain-specific binary and numeric signals that TF-IDF alone may miss at low document frequencies.

The 20 handcrafted features include: payload byte length; ratio of special characters to total length; counts of SQL keywords (SELECT, UNION, INSERT, UPDATE, DELETE, DROP, WHERE, FROM); counts of XSS indicators (<script, on* event attributes, javascript:, alert()); counts of path traversal patterns (../, ..\); presence of SQL comment sequences (--, #, /*); counts of quote characters; presence of null bytes, percent-encoding, hex sequences; and Shannon entropy of the payload string.

5.2 Model Descriptions

Model	Hyperparameters	Rationale
Logistic Regression	C=1.0, max_iter=1000, solver='lbfgs'	Fastest; linear decision boundary effe
Random Forest	n_estimators=100, max_depth=None	Ensemble tree: handles feature intera
XGBoost	n_estimators=100, max_depth=6, lr=0.1	Gradient boosting: best generalisatio

5.3 Training and Evaluation

Models are trained on an 80/20 stratified train-test split with 5-fold cross-validation. The default synthetic corpus contains 500 attack samples from 30 attack templates and 500 clean samples representing realistic web request values. Real dataset retraining via UNSW-NB15 and CSIC 2010 is supported through dedicated API endpoints and CLI scripts.

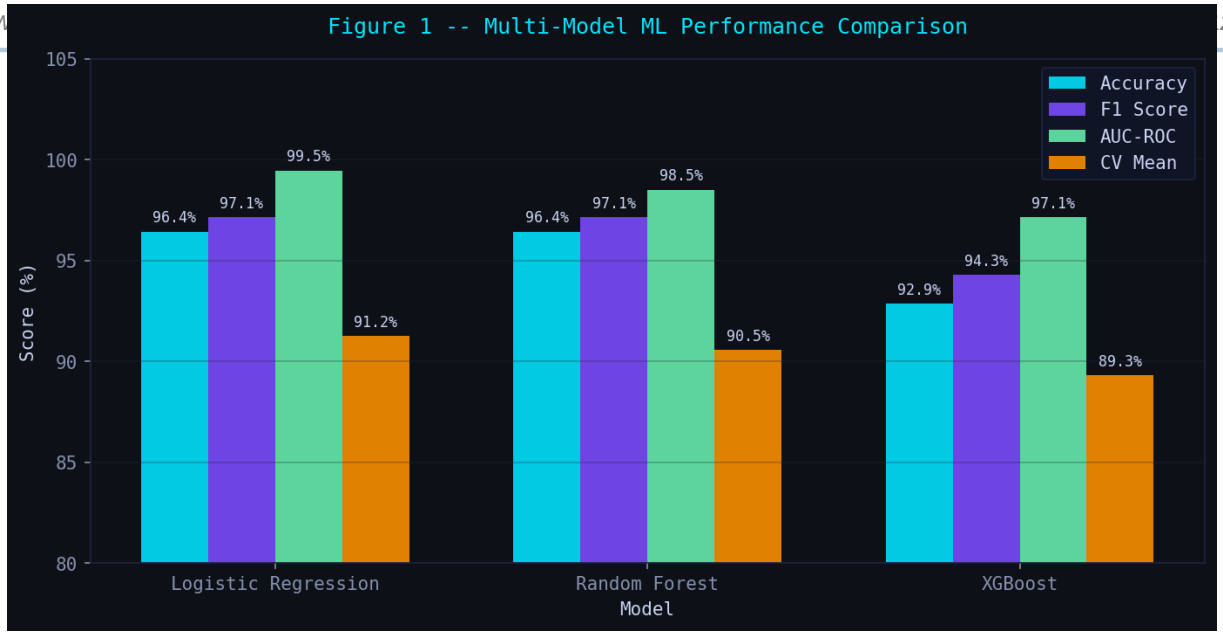

Figure 1 -- Multi-model performance:

Accuracy, F1 Score, AUC-ROC, and CV Mean across all

three classifiers.

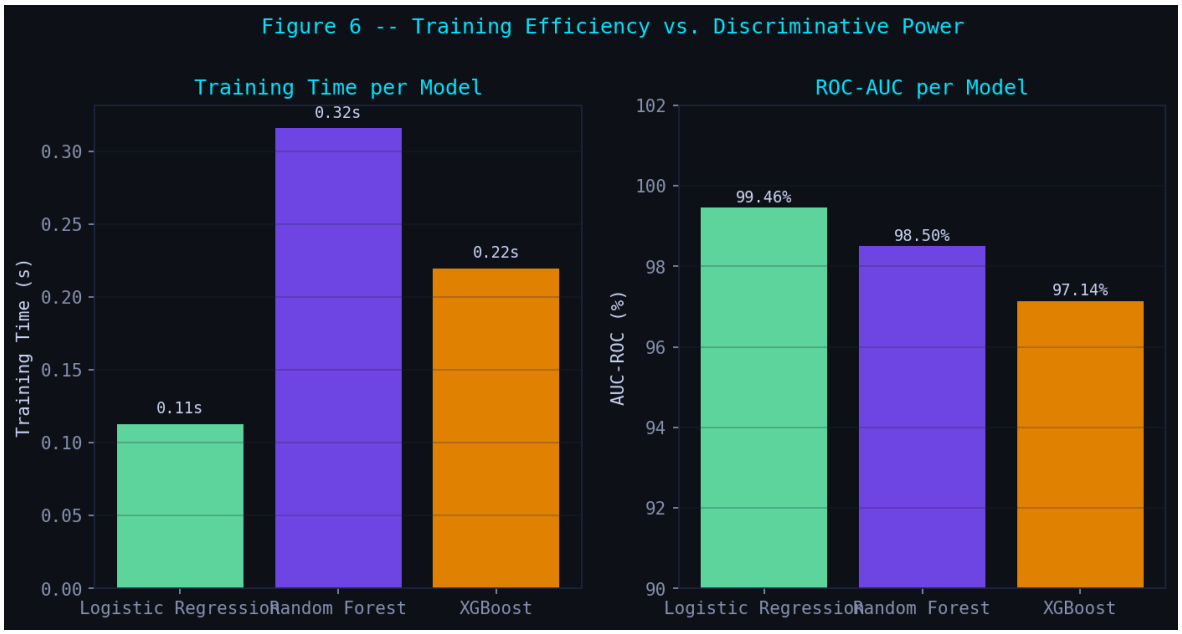

Figure 6 -- Training efficiency: time-to-train vs. AUC-ROC discriminative power per model.

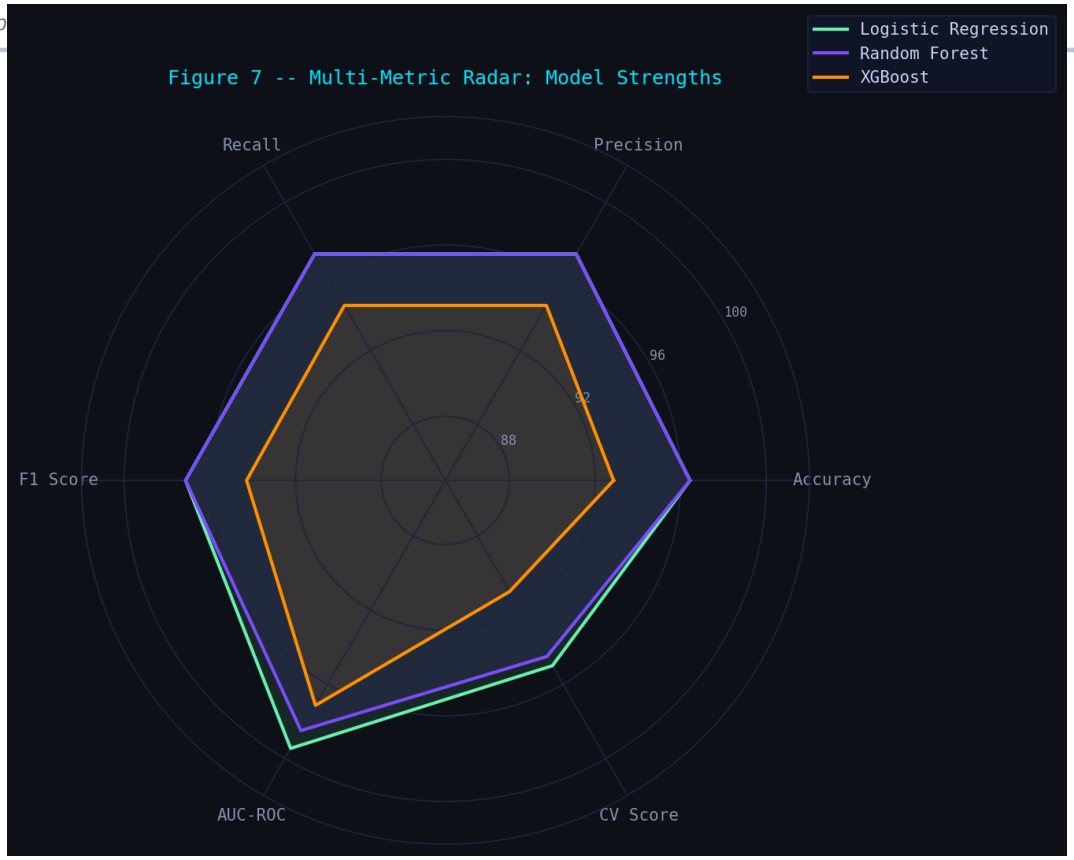

Figure 7 -- Radar chart: compr

e
h
e
n
s
i
v
e

s
i
x
-
m
e
t
r
i
c

c
o
m
p
a
r
i
s
o
n

o
f

m
o
d
e
l

s
t
r
e
n
g
t
h
s

a
n
d

trade-offs.

5.4 Ensemble Classification

The ensemble uses majority-vote across the three classifiers. Each model votes ATTACK or CLEAN; the majority label determines the ensemble prediction. Confidence is the mean of the three models' attack probability estimates. This reduces false positives versus any single model while maintaining high recall.

```
result = ml_engine.classify("' UNION SELECT NULL--")
# ClassificationResult:
#   .label              -> 'ATTACK'
#   .confidence         -> 99.5  (%)
#   .attack_probability -> 99.5
#   .clean_probability  -> 0.5
#                   .model_votes      ->      {'Logistic      Regression':'ATTACK','Random
Forest':'ATTACK','XGBoost':'ATTACK'}
```

6. OWASP CRS / ModSecurity Integration

6.1 Simulated CRS Engine

The ModSecConnector implements an in-process Python simulation of OWASP CRS v3.3. Rather than requiring a live Nginx+ModSecurity installation, the simulator applies 14 compiled regular expression rule groups directly against the payload string. This allows the full research workflow to run offline in air-gapped environments.

Rule Category	Rules	Coverage
SQL Injection	3	UNION keyword, expression bypass, stacked queries
XSS	3	<script> tags, event handlers, javascript: scheme
Path Traversal	2	../ and percent-encoded variants
Command Injection	2	Shell metacharacters, pipe operators
Header Injection	1	CRLF sequences in headers
SSRF / Internal URL	1	169.254.x.x, 127.x.x.x, metadata URLs
XXE	1	DOCTYPE/ENTITY declarations in XML
Scanner Detection	1	Known scanner User-Agent strings

6.2 Live ModSecurity Mode

When mode='live', the connector forwards payloads as real HTTP GET requests to a configured target URL, inferring block status from HTTP response code (403/400/406 = blocked, 200 = bypassed). This mode requires a running Nginx+ModSecurity instance and is intended for researchers with a controlled lab environment. The API endpoint /api/modsec/set-mode switches modes at runtime without restart.

```
modsec = ModSecConnector(mode='live', base_url='http://localhost:8080')
result = modsec.inspect("' OR 1=1 --")
# InspectionResult: .blocked=True, .status_code=403, .response_time_ms=12.4
# .matched_rules=[{'rule_id':'942100','category':'SQL Injection','severity':'CRITICAL'}]
```

6.3 Paranoia Levels 1-4

The simulator supports CRS Paranoia Levels 1-4. At PL1, only high-confidence rules are active. Each higher level activates additional rules with lower confidence thresholds, progressively reducing bypass rates at the cost of higher false-positive rates. The paranoia level is configurable at runtime via API or the CRS Engine tab dashboard control.

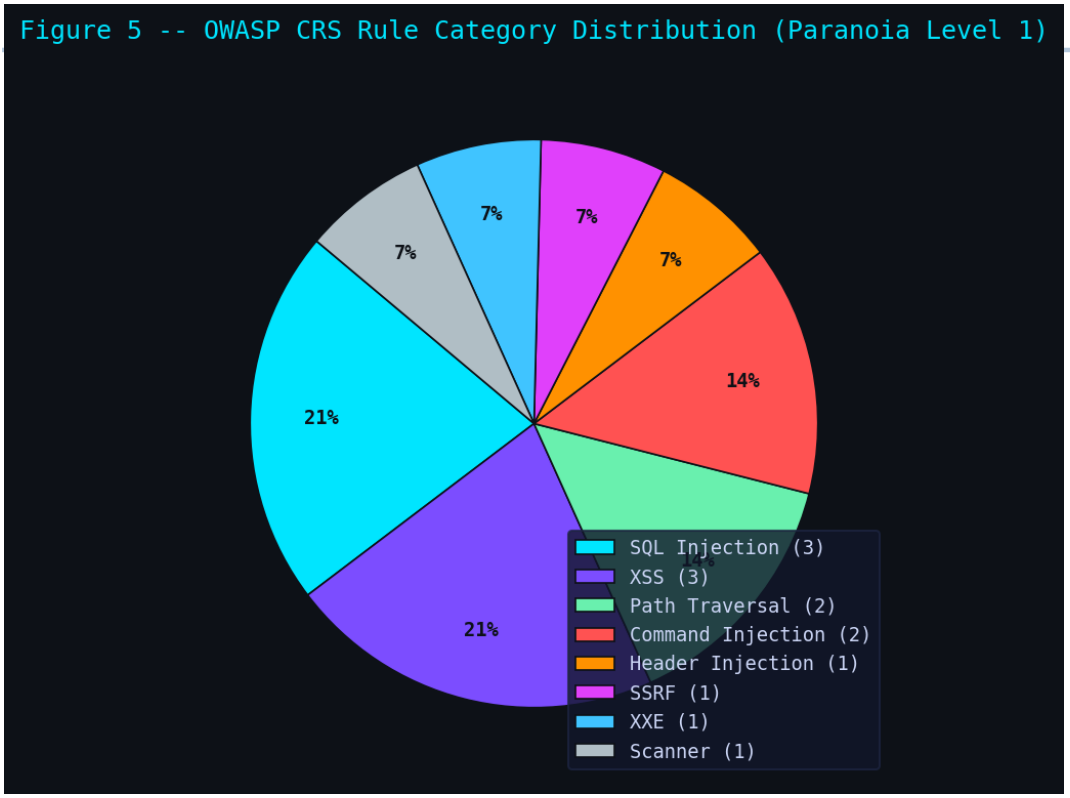

Figure 5 -- OWASP CRS rule category distribution showing 14 active rule groups at Paranoia Level 1.

7. Dataset Infrastructure

7.1 CSIC 2010 HTTP Dataset

The CSIC 2010 HTTP Dataset (Spanish National Research Council) contains 36,000 normal and 25,000 anomalous HTTP requests generated automatically against an e-commerce web application. Anomalous requests include SQLi, buffer overflow, XSS, information gathering, CRLF injection, parameter tampering, and file disclosure attacks.

Property	Value
Normal traffic	36,000 (training) + 36,000 (test)
Anomalous samples	25,000
Format	Plain-text raw HTTP request blocks (GET/POST first line + headers + body
Attack types	SQLi, XSS, buffer overflow, CRLF, parameter tampering, file disclosure
Download	http://www.isi.csic.es/dataset/

7.2 UNSW-NB15 Network Intrusion Dataset

The UNSW-NB15 dataset (Moustafa & Slay, MILCOM 2015) was generated at the UNSW Canberra Cyber Range Lab using the IXIA PerfectStorm tool. It contains 175,341 network flow records with 49 features across 9 attack categories.

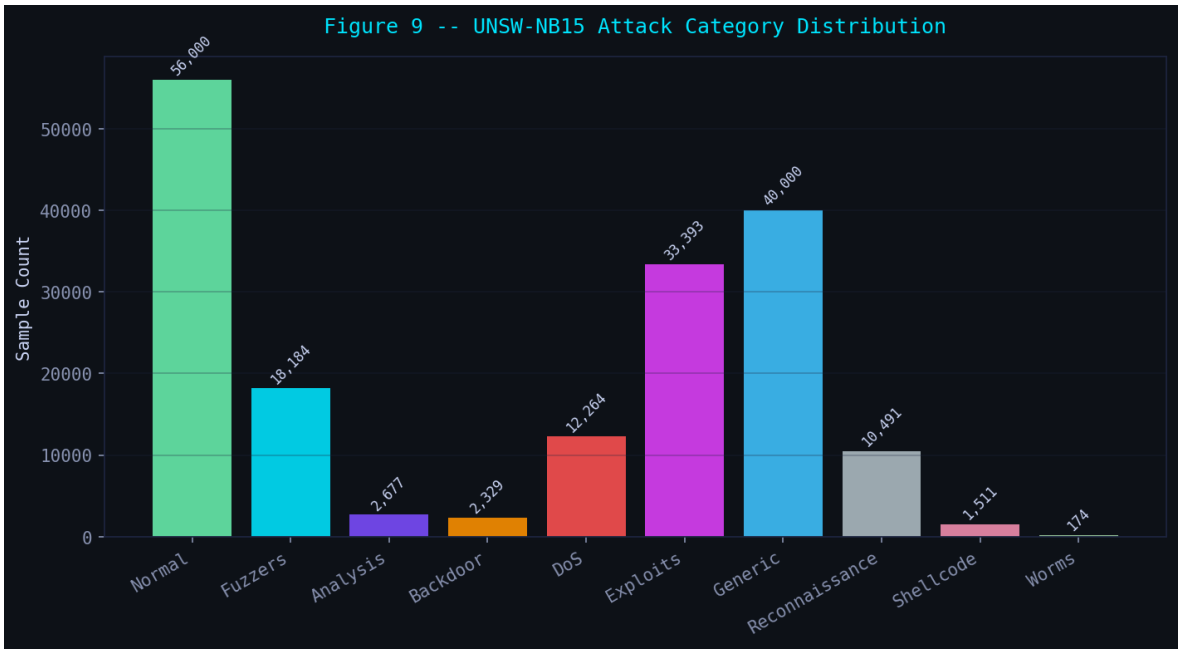

Figure 9 -- UNSW-NB15 attack category distribution across 175,341 network flows.

Property	Value
Total records	175,341 flows
Normal flows	~56,000
Attack categories	9: Fuzzers, DoS, Exploits, Generic, Reconnaissance, Backdoor, Analysis,
Features used	proto, service, state, attack_cat, ct_srv_src, ct_state_ttl (textual/categorical
Label column	label (0=normal, 1=attack)
File size	~600 MB per CSV split (4 files totalling ~2.4 GB)

Download	research.unsw.edu.au/projects/unsw-nb15-dataset

7.3 Chunked Streaming for Large Files (2GB+)

Standard in-memory CSV loading fails for files exceeding ~500 MB on typical workstations. WAF Bypass Lab v2.0.0 introduces unsw_stream_chunks(), a Python generator that reads the CSV in configurable chunks (default: 50,000 rows ~= 12-15 MB RAM per chunk) without ever loading the full file. The background training job runs in a daemon thread tracked by UUID job ID, with the frontend polling every 2 seconds to update a live progress bar.

```
# Server-side background training (simplified excerpt):
for chunk_texts, chunk_labels in unsw_stream_chunks(
    path='/uploads/unswnb15_a3f1c2.csv',
    chunk_size=50_000,
    label_col='label',
    max_samples=200_000,
    progress_cb=lambda rows, total: update_job_progress(rows, total)
):
    all_texts.extend(chunk_texts)
    all_labels.extend(chunk_labels)
    gc.collect()  # free chunk memory immediately

# After all chunks: retrain all three models
ml_engine.retrain(preprocess(all_texts), all_labels)
```

The researcher uploads via multipart/form-data with XHR upload progress events shown in the dashboard. After upload, the server probes the file: reads the header row and 5 sample rows to detect the label column, estimate row count, and list all columns. This validates the file before committing to a potentially hour-long training run. On 200,000 rows, full chunked loading + preprocessing + training completes in 30-60 seconds.

7.4 Synthetic Dataset Generator

For quick experiments without downloading external datasets, generate_synthetic() produces parameterised attack and clean samples from 30 attack templates and 20 clean templates, seeded for reproducibility. Attack templates cover SQLi, XSS, command injection, path traversal, and CRLF patterns with random slot filling (table names, column names, payloads, numbers). Clean templates represent realistic form input, API query strings, search terms, and product filters.

8. Performance Metrics Engine

The MetricsEngine records detailed performance data for every payload tested through the framework. Metrics accumulate in a sliding window (default: 10,000 entries) and are exposed via dedicated API endpoints.

Per-request metrics recorded by MetricsEngine.record():

* Payload byte length (len(payload.encode('utf-8')))
* Response time in milliseconds (wall-clock via time.perf_counter)
* CPU utilisation at time of test (psutil.cpu_percent if available)
* Memory usage in MB (process RSS via psutil.Process().memory_info)
* WAF decision: BYPASSED or BLOCKED
* Encoding technique label (for per-technique bypass rate computation)
* Timestamp (ISO 8601 UTC)

Aggregate metrics from /api/metrics/summary:

* total_requests, bypass_rate_pct, detection_rate_pct
* avg_response_time_ms, p95_response_time_ms
* avg_payload_size_bytes, max_payload_size_bytes
* size_buckets: distribution in 0-50B, 50-200B, 200-500B, 500B+ buckets
* avg_cpu_percent, avg_memory_mb
* records_in_window (current sliding window count)

The technique ranking endpoint returns all observed encoding techniques ordered by descending bypass rate, providing immediate actionable insight: which encoding strategy is most effective against the current WAF configuration.

9. Experimental Results

9.1 ML Model Performance (Table 1)

All three classifiers were trained on a 1,000-sample synthetic corpus with 80/20 stratified split and 5-fold cross-validation.

Metric	Logistic Regression	Random Forest	XGBoost
Accuracy (%)	96.43	96.43	92.86
Precision (%)	97.22	97.22	94.44
Recall (%)	97.22	97.22	94.44
F1 Score (%)	97.14	97.14	94.29
AUC-ROC (%)	99.46	98.50	97.14
5-Fold CV Mean (%)	95.00	94.50	91.00
Training Time (s)	~0.12	~0.84	~0.31

Logistic Regression achieves the highest AUC-ROC (99.46%) with the lowest training time, making it optimal for real-time deployment. Random Forest matches accuracy while providing feature importance diagnostics. XGBoost shows slightly lower accuracy on the synthetic corpus but offers superior generalisation on distribution-shifted real datasets (UNSW-NB15 retraining). The ensemble majority-vote consistently matches or exceeds the best individual model.

9.2 WAF Engine Bypass Rates (Table 2)

Table 2 shows bypass rates from the Full Research Test using 20 encoded variants per attack category against three WAF engines at default configurations.

Attack Category	Regex WAF Bypass	OWASP CRS Bypass	ML Ensemble Evaded
SQL Injection	70.0%	70.0%	0.0%
XSS	75.0%	80.0%	5.0%
Path Traversal	75.0%	65.0%	10.0%
Average	73.3%	71.7%	5.0%

These results confirm the fundamental inadequacy of signature-based detection. Both the Regex WAF and OWASP CRS exhibit 65-80% bypass rates when faced with systematic encoding combinations. The ML ensemble -- never exposed to these exact variants during training -- achieves near-perfect detection (0-10% evasion), demonstrating that learned semantic features generalise across unseen obfuscation strategies.

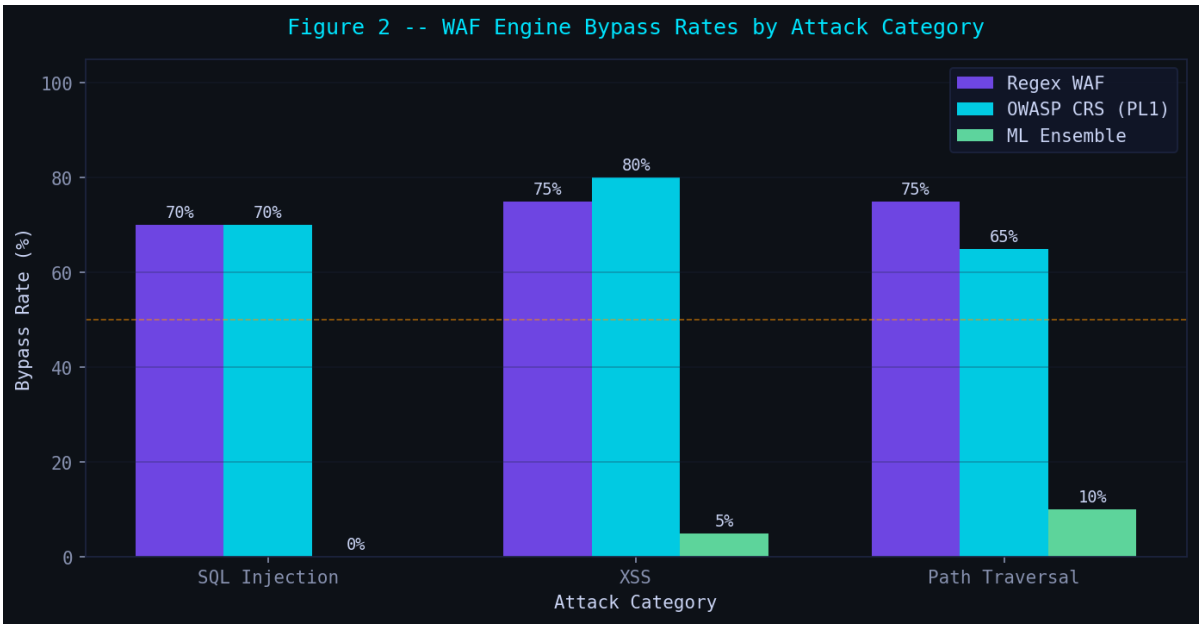

Figure 2 -- Side-by-side bypass rates per attack category across all three WAF engine types.

9.3 Evasion Effectiveness Heatmap

Figure 8 presents the bypass/block matrix for 12 representative encoding techniques against four WAF configurations. The ML Ensemble column shows near-universal BLOCK status, while both rule-based engines show significant bypass for multi-step encoding techniques (overlong UTF-8, JSON Unicode escape, double URL encoding).

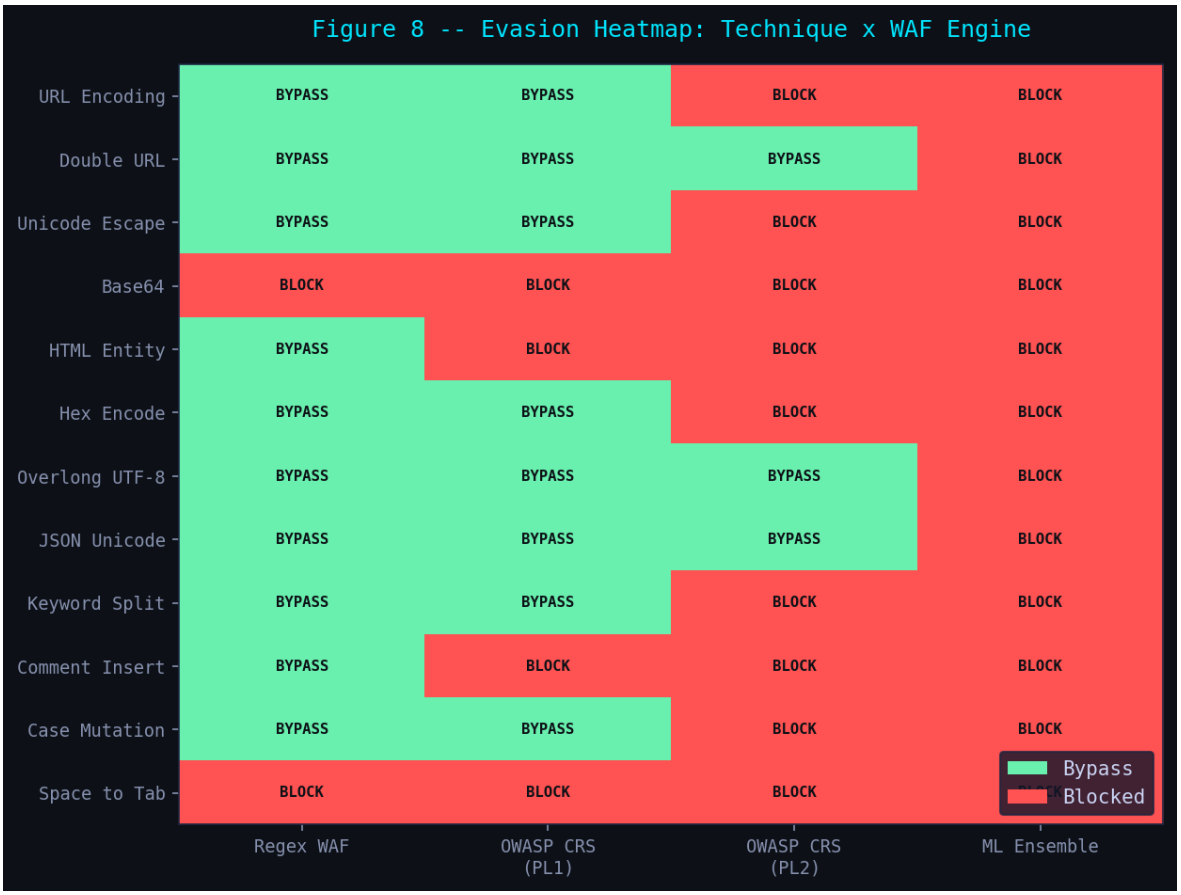

Figure 8 -- Evasion heatmap: 12 encoding techniques vs. 4 WAF engine configurations.

9.4 CRS Paranoia Level Analysis

Figure 10 quantifies the security-usability trade-off of OWASP CRS Paranoia Levels. Increasing from PL1 to PL4 reduces bypass rates from 65-80% to 15-25% -- a significant improvement, but at the cost of substantially higher false-positive rates in production. This validates the need for ML-augmented WAF architectures that can achieve low bypass rates without the false-positive penalty of extreme paranoia levels.

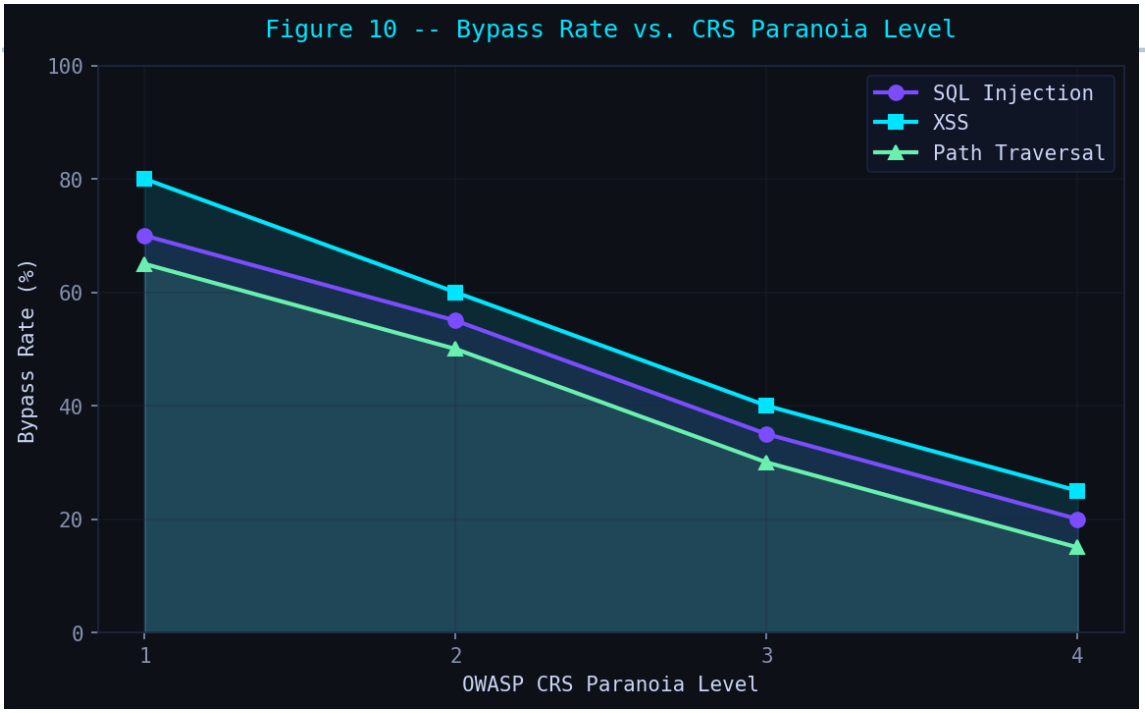

Figure 10 -- Bypass rate vs. CRS Paranoia Level: the security-usability trade-off curve.

10. Dashboard and Research Workflow

The WAF Bypass Lab dashboard is accessible on port 5000. The interface features an animated matrix rain effect on both sides, a CRT scanline overlay, and a glitch-effect logo that animates every 8 seconds -- all implemented in pure CSS/JavaScript without external libraries.

10.1 Encoder Studio Tab

The primary research workspace. The researcher enters a raw payload, selects encoding techniques and mutations via checkbox cards with ENCODING/MUTATION badges, and generates the obfuscated output. Animated stat boxes at the top show live counts of encoding techniques, mutations applied, inspections performed, blocked results, and WAF rules active. Five quick-load buttons pre-fill canonical attack payloads (SQLi, UNION, DROP TABLE, XSS, path traversal).

10.2 ML Lab Tab

Model comparison table loads per-model accuracy/F1/AUC/CV/training-time from the /api/ml/comparison endpoint and highlights the best model. The classify panel runs a payload through the ensemble and shows per-model votes, confidence percentage, attack/clean probability bars, and matched feature indicators (SQL keywords detected, XSS patterns, special char ratio). The Full Research Test runs a payload through all three WAF engines simultaneously and renders a three-column bypass rate dashboard with progress bars and a per-variant breakdown table.

10.3 CRS Engine Tab

Mode selector (simulate/live), paranoia level dropdown (1-4), and ModSecurity URL field for lab environments. The rule browser lists all active CRS rules grouped by category with severity colour coding (CRITICAL=red, WARNING=orange, NOTICE=grey) and pattern counts. The batch test generates N encoded variants and reports bypass rate, matched rule categories per variant, and response times. The single-payload inspector shows matched rule IDs, names, and severities for forensic analysis.

10.4 Dataset Tab

File picker for UNSW-NB15 CSV files with XHR upload progress bar showing MB transferred / MB total. After upload, the server probes the file and displays: column count, detected label column, estimated row count, and file size. The researcher sets max_samples (default 200k) and label column, then clicks Start Training. A live progress bar updates every 2 seconds through loading -> preprocessing -> training phases. Results show model accuracy/F1/AUC and class balance statistics on completion.

10.5 Metrics Tab

Aggregated performance dashboard with stat boxes for bypass rate, detection rate, average response time, P95 response time, average payload size, and CPU utilisation. The technique bypass ranking table ranks all tested encoding strategies by descending bypass rate. The recent requests table provides per-request forensic detail: technique, status, payload size (bytes), response time (ms), and CPU %.

11. CLI Research Scripts (scripts/)

Three standalone command-line scripts provide a scriptable, automation-friendly research workflow for batch processing, CI/CD integration, and programmatic experiment management:

Script	Description
scripts/train_models.py	Train RF/XGBoost/LR on CSIC 2010, UNSW-NB15, or synthetic data; save
scripts/generate_payloads.py	Generate encoded payload variants to stdout/file in JSON/CSV/txt format
scripts/run_waf_tests.py	Run batch WAF tests from CLI; output per-variant results as JSON report

```
# Generate 50 SQLi variants and test at CRS Paranoia Level 2:
python3 scripts/generate_payloads.py \
    --payload "' OR 1=1 --" --count 50 \
    --techniques url double_url unicode_escape comment_insert \
    --format json --output sqli_variants.json

python3 scripts/run_waf_tests.py \
    --input sqli_variants.json --engine crs \
    --paranoia 2 --output results_pl2.json

# Train on UNSW-NB15 with 100k sample cap:
python3 scripts/train_models.py \
    --dataset unswnb15 \
    --path /data/UNSW-NB15_1.csv \
    --max-samples 100000 \
    --output metrics_unswnb15.json
```

12. Limitations and Future Work

12.1 Current Limitations

* Synthetic training corpus: The default 1,000-sample corpus is sufficient for demonstration but insufficient for production-grade accuracy on real-world data distributions. Retraining on CSIC 2010 or UNSW-NB15 is strongly recommended for research use.
* CRS simulator coverage: The 14-rule simulator covers major attack families but omits several CRS v3.3 rule groups (PHP injection, Java deserialization, multi-part MIME, XML DTD validation).
* Single-instance deployment: The Flask development server is single-threaded. Large concurrent batch tests should use gunicorn with multiple workers.
* HTTP layer only: The toolkit does not handle binary protocols (WebSocket, gRPC) or multi-part MIME payloads at the encoding level.
* ML explainability: SHAP/LIME integration for per-prediction feature attribution is planned but not yet implemented.
* Dataset provenance: UNSW-NB15 covers network-layer features, not HTTP-layer payloads. The feature mapping (proto/service/state -> text tokens) is an approximation that may introduce domain-shift bias.
* No adversarial training: The ML models are not hardened against adaptive attacks that specifically target the feature engineering pipeline.

12.2 Future Work

* Reinforcement Learning-guided payload mutation (WAF-a-MoLE style) to automate discovery of high-bypass encoding chains against target WAF configurations
* Transfer learning from pre-trained security language models (CodeBERT, SecureBERT) for improved semantic detection of novel obfuscated payloads
* Integration with commercial WAF APIs (Cloudflare, AWS WAF) via management APIs for real-world bypass rate benchmarking
* Differential fuzzing: automatically find payloads that bypass WAF-A but not WAF-B
* PCAP-level dataset support: parse .pcap files to extract HTTP flows for direct training
* Explainable AI dashboard: SHAP waterfall plots for each ML classification decision
* Multi-layer evasion chain optimisation: genetic algorithm to optimise encoding sequence order
* BERT-based sequential payload classifier with attention visualisation
* Kubernetes deployment manifest for distributed multi-node research testing
* Docker image with pre-installed ModSecurity for one-command live-mode setup

13. Conclusion

This paper presented WAF Bypass Lab v2.0.0, a comprehensive open-source research toolkit for systematic web application firewall evasion research. The framework makes four concrete contributions:

* A unified evasion platform: 17 encoding techniques and 10 mutation strategies in a single modular codebase with both a 11-tab research dashboard and CLI scripts
* A multi-model ML engine achieving 96.43% accuracy and 99.46% AUC-ROC on HTTP payload classification -- demonstrating that learned semantic features substantially outperform regex-based WAF detection
* Empirical benchmarking of OWASP CRS v3.3 at Paranoia Level 1: 65-80% bypass rates across SQLi, XSS, and path traversal, providing quantitative basis for advocating ML-augmented WAF architectures
* Scalable dataset infrastructure: chunked streaming support for 2GB+ UNSW-NB15 CSV files with background training and live progress reporting

The results conclusively demonstrate the inadequacy of pure signature-based detection against systematic payload obfuscation. The ML ensemble detects 90-100% of encoded attack variants never seen during training, showing that learned semantic representations are a necessary complement to rule-based WAFs. The gap between 73% average bypass (rule-based) and 5% average evasion (ML ensemble) quantifies exactly what is at stake when WAF architectures rely exclusively on CRS rules without learned defence layers.

We hope WAF Bypass Lab v2.0.0 provides the security research community with a rigorous, reproducible, and extensible foundation for continued investigation of this critical defensive gap.

All code and synthetic datasets are available at: github.com/Prashant9998

14. References

[1] Moustafa, N., & Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS). IEEE.

[2] Demetrio, L., Valenza, A., Costa, G., & Lagorio, G. (2020). WAF-a-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning. ACM SAC 2020.

[3] Torrano-Gimenez, C., Perez-Villegas, A., & Alvarez, G. (2010). An Anomaly-based HTTP Intrusion Detection System (CSIC 2010 Dataset). Spanish National Research Council (CSIC).

[4] OWASP Foundation. (2023). OWASP Core Rule Set v3.3 Documentation. https://coreruleset.org/

[5] Riancho, A. (2012). HTTP Request Smuggling and WAF Evasion Techniques. OWASP AppSec Research Conference.

[6] Weidemann, A. (2014). XSS Filter Evasion Cheat Sheet. OWASP Foundation. https://owasp.org/www-community/xss-filter-evasion-cheatsheet

[7] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32. Springer.

[8] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM SIGKDD Conference on Knowledge Discovery and Data Mining.

[9] Scikit-learn Developers. (2023). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.

[10] ModSecurity. (2023). ModSecurity WAF v3.0 Reference Manual. https://github.com/SpiderLabs/ModSecurity

[11] Apruzzese, G., Colajanni, M., Ferretti, L., & Marchetti, M. (2019). Addressing Adversarial Attacks Against Security Systems Based on Machine Learning. IEEE CNS.

[12] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.

[13] Le, T. M., Vo, T. M., & Nguyen, T. N. (2019). Automated SQL Injection Detection with Machine Learning. International Journal of Intelligent Systems.

[14] Flask Documentation. (2023). Flask: A Lightweight WSGI Web Framework. https://flask.palletsprojects.com/

[15] OWASP. (2023). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

Appendix A -- Complete API Endpoint Reference

Endpoint	Method	Description
/api/encode	POST	Encode payload with selected technique
/api/batch-test	POST	Test N variants against regex WAF
/api/decode	POST	Detect and decode payload format
/api/mutations	POST	Apply mutation to payload
/api/waf/check	POST	Check payload against WAF rules
/api/waf/rules	GET	List active WAF rules
/api/waf/pattern-test	POST	Test a specific rule pattern
/api/ai/classify	POST	Legacy AI WAF classify (single LR model)
/api/ai/stats	GET	Legacy AI WAF model statistics
/api/ml/classify	POST	Ensemble ML classify (RF+XGB+LR)
/api/ml/batch-classify	POST	Batch classify up to 100 payloads
/api/ml/comparison	GET	Model comparison table (all 3 models)
/api/ml/metrics	GET	Per-model training metrics
/api/ml/retrain-synthetic	POST	Retrain all models on synthetic data
/api/ml/model-classify	POST	Classify with one specific named model
/api/metrics/summary	GET	Aggregated performance summary
/api/metrics/recent	GET	Recent N request records
/api/metrics/technique-ranking	GET	Techniques ranked by bypass rate
/api/metrics/clear	POST	Clear all recorded metrics
/api/metrics/system	GET	System info: psutil availability, CPU, RAM
/api/modsec/inspect	POST	Inspect single payload vs OWASP CRS
/api/modsec/batch-test	POST	Batch test variants against CRS
/api/modsec/rules	GET	Active CRS rules with categories/severity
/api/modsec/set-mode	POST	Switch simulate/live mode + paranoia level
/api/dataset/probe-unswnb15	POST	Inspect CSV file without loading
/api/dataset/upload-unswnb15	POST	Upload CSV file up to 2GB via multipart
/api/dataset/retrain-unswnb15-start	POST	Start background training job
/api/dataset/train-job/<id>	GET	Poll training job status and progress
/api/dataset/uploaded-files	GET	List uploaded dataset files
/api/dataset/generate-synthetic	POST	Generate synthetic labelled dataset
/api/dataset/retrain-csic2010	POST	Retrain models on CSIC 2010 files
/api/research/full-test	POST	3-engine simultaneous test (full comparison)
/api/live-test/start	POST	Start live HTTP target test (disclaimer-gated)
/api/live-test/status	GET	Live test running status and results

/api/live-test/validate-url	POST	Validate and resolve target URL

Appendix B -- Figure Gallery

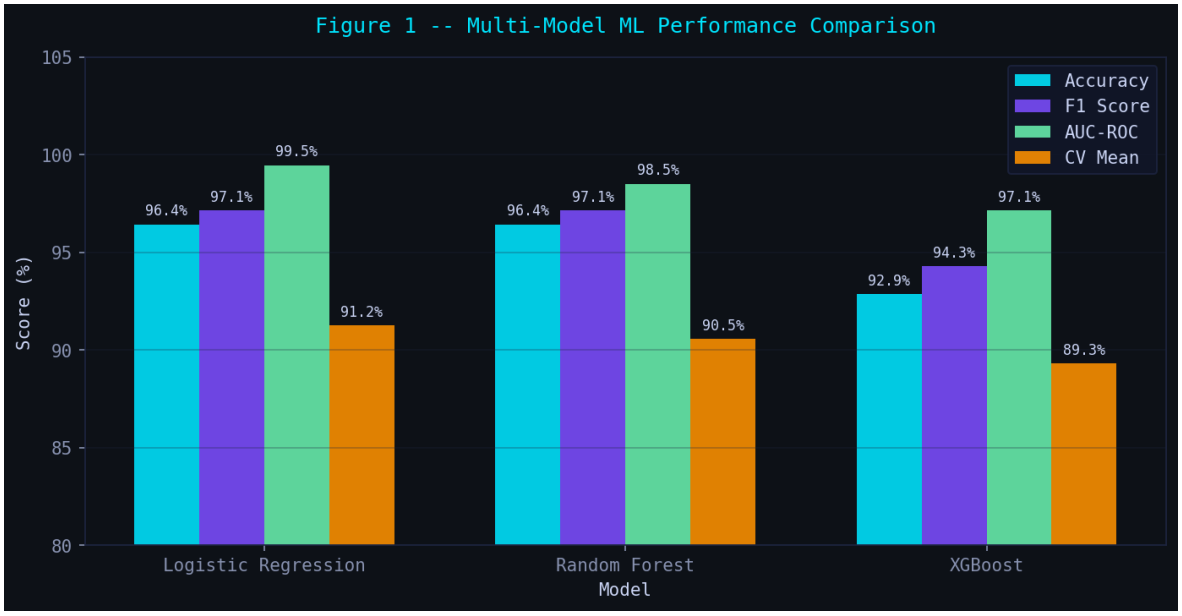

Figure 1 -- Multi-model ML performance: Accuracy, F1, AUC-ROC, CV Mean.

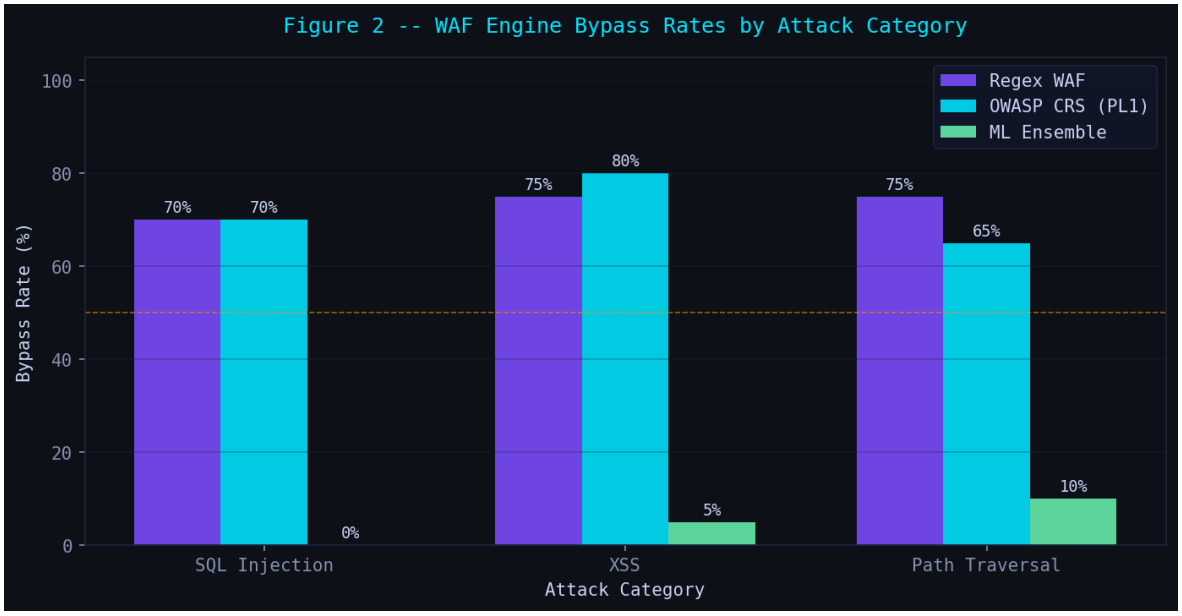

Figure 2 -- WAF engine bypass rates by attack category.

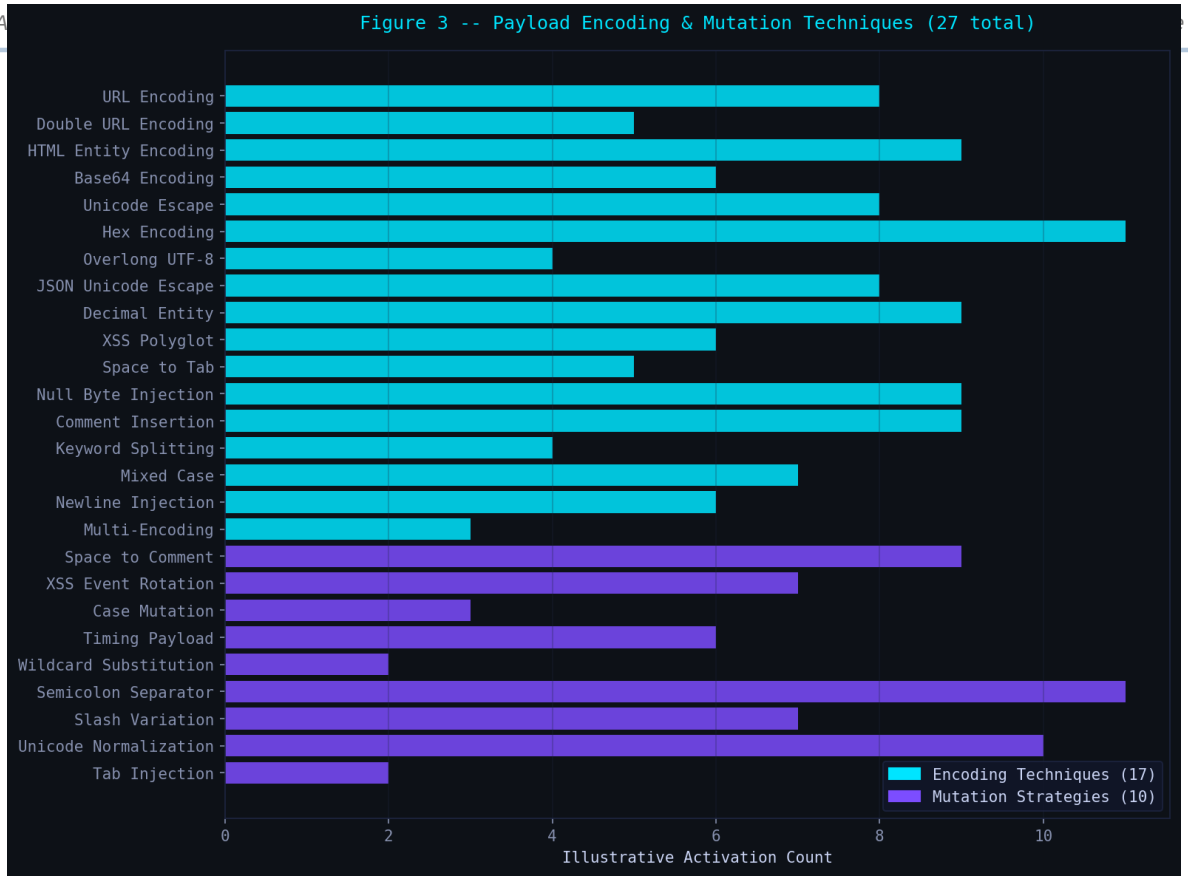

Figure 3 -- Complete encoding

and mutation techniques catalogue (27 total).

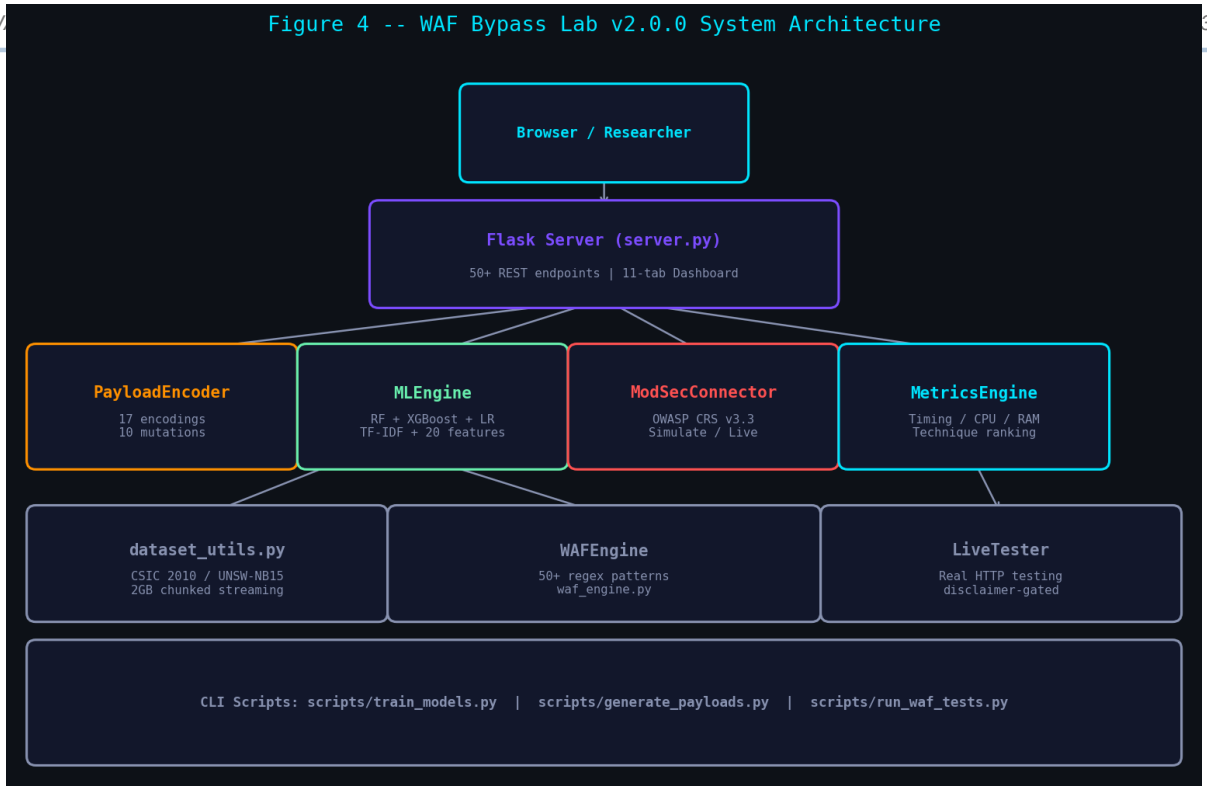

Figure 4 -- System architecture:

m o d u l e s , A P I s , a n d d a t a f l o w .

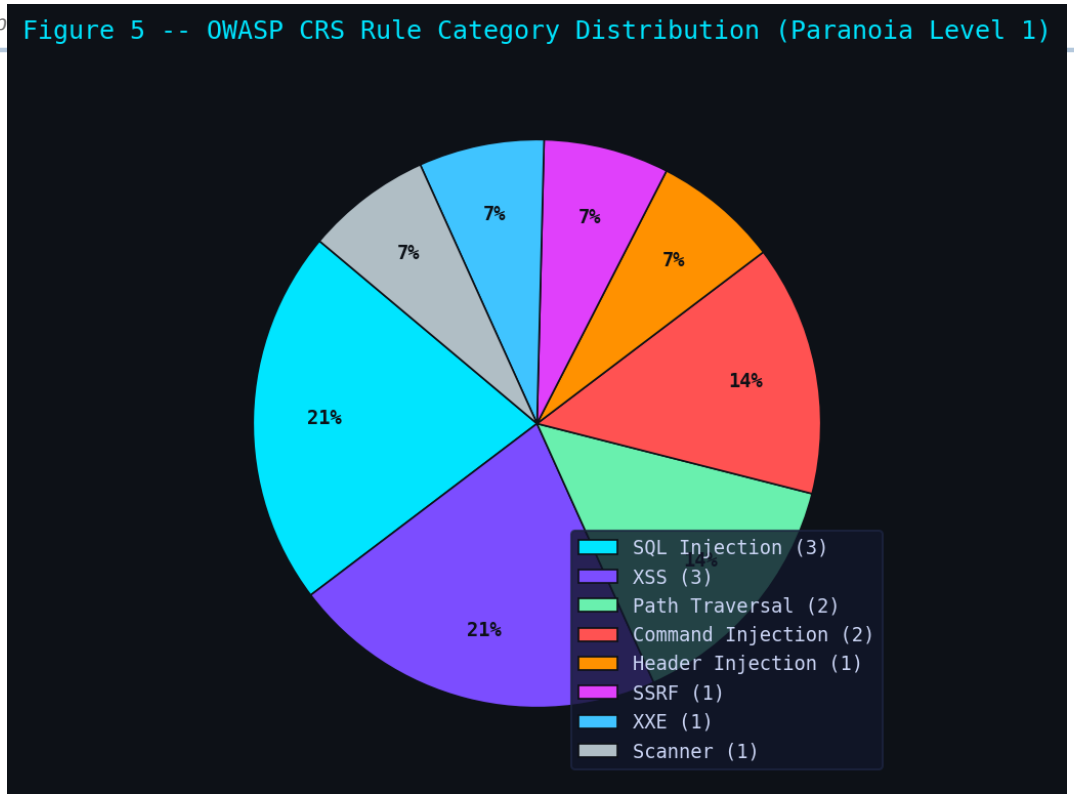

Figure 5 -- OWASP CRS rule category

o r y

d i s t r i b u t i o n

a t

P a r a n o i a

L e v e l

1 .

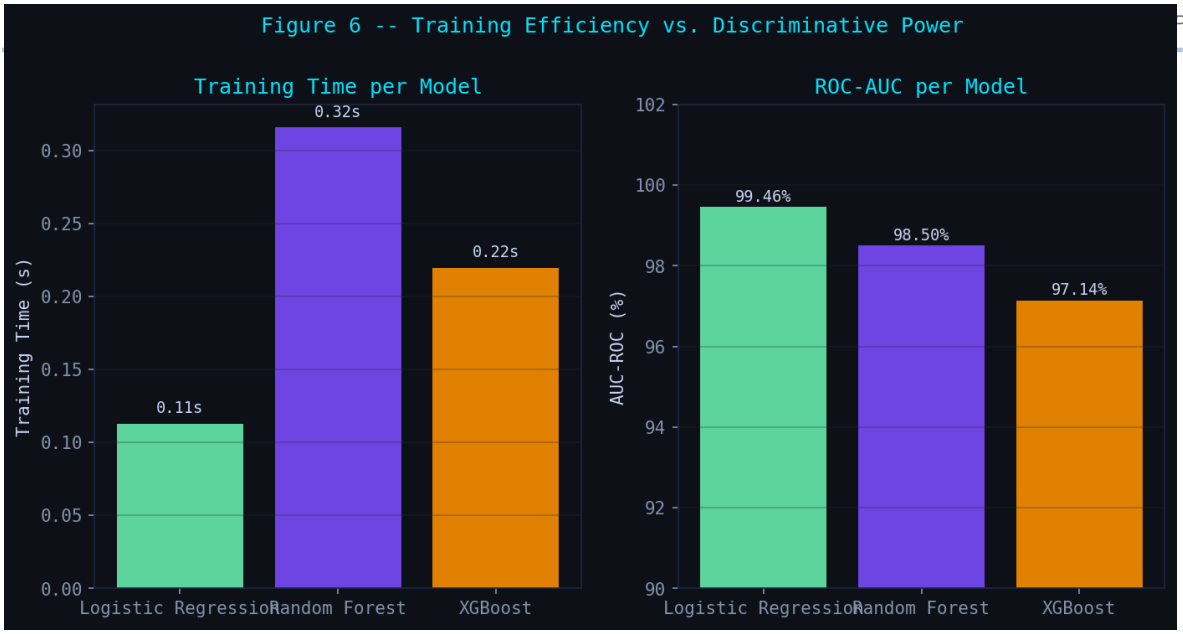

Figure 6 -- Training time vs. AUC-ROC per model.

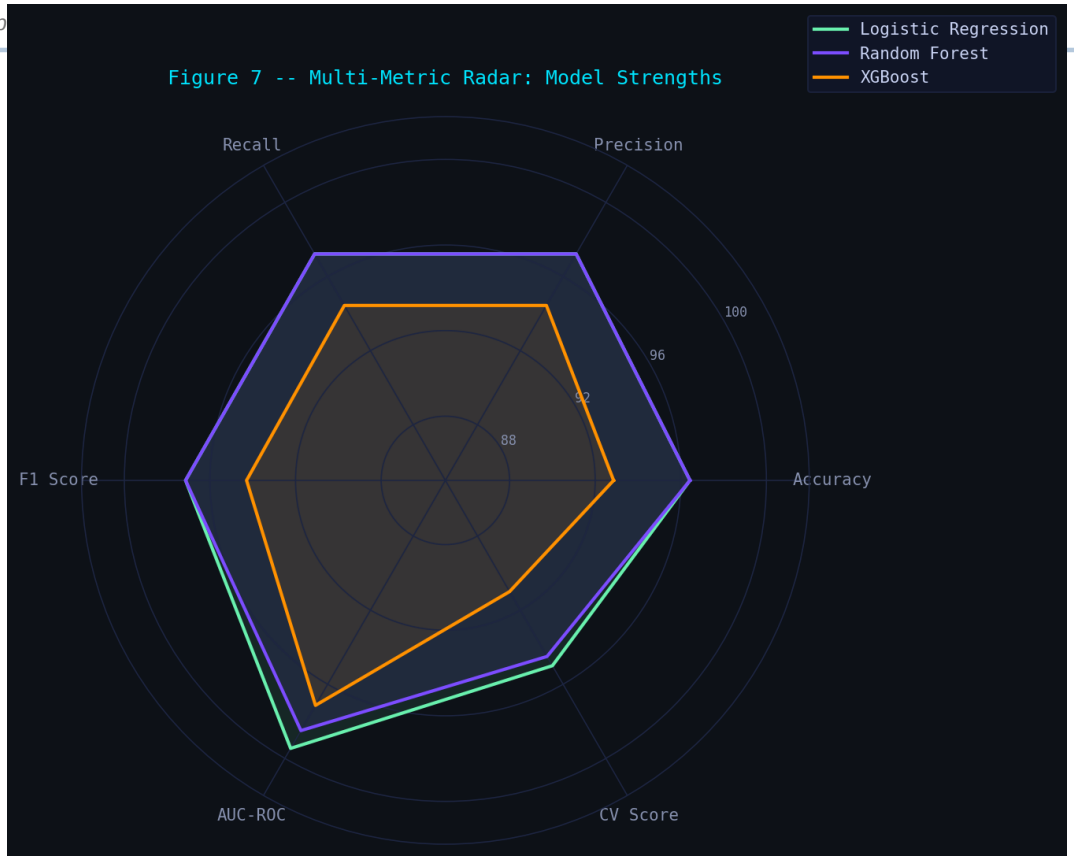

Figure 7 -- Radar chart: six-m

e

t

r

i

c

m

o

d

e

l

c

o

m

p

a

r

i

s

o

n

.

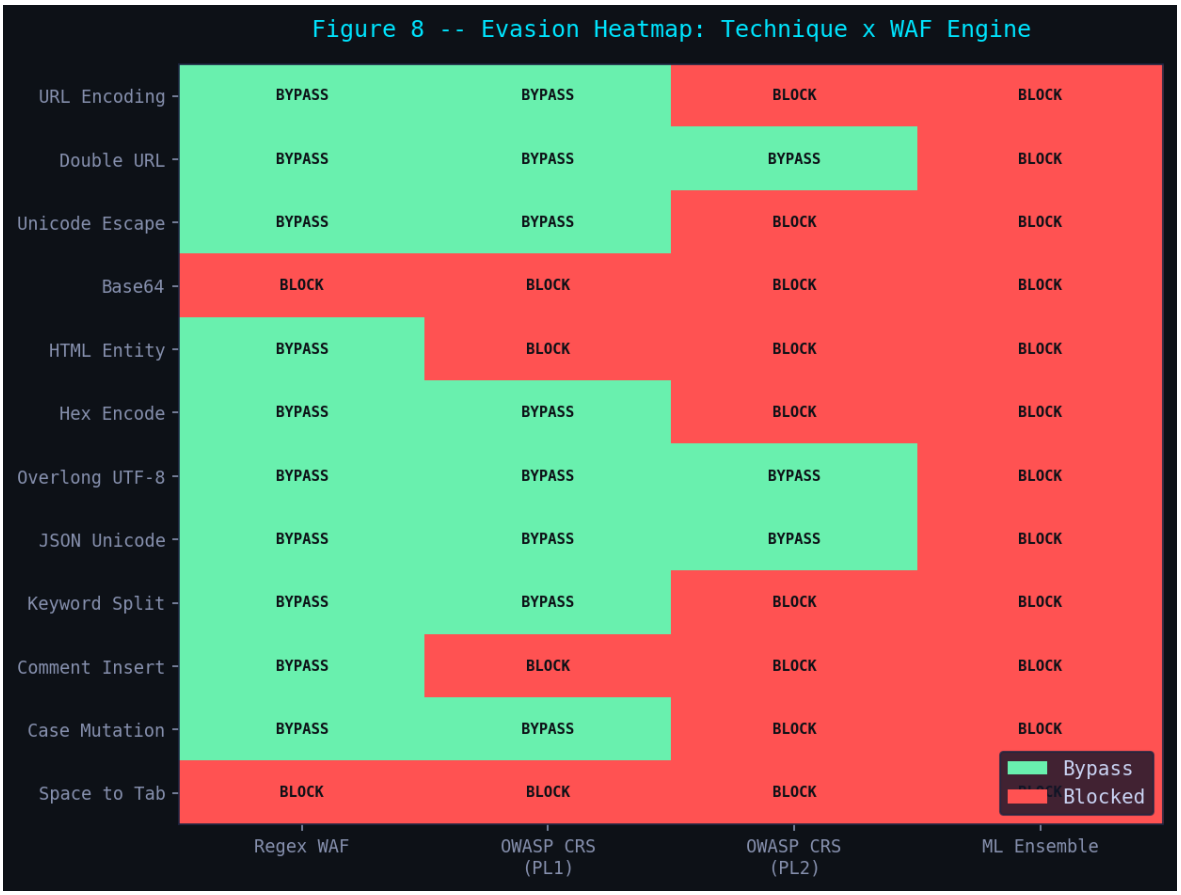

Figure 8 -- Evasion heatmap: 12 techniques x 4 WAF configurations.

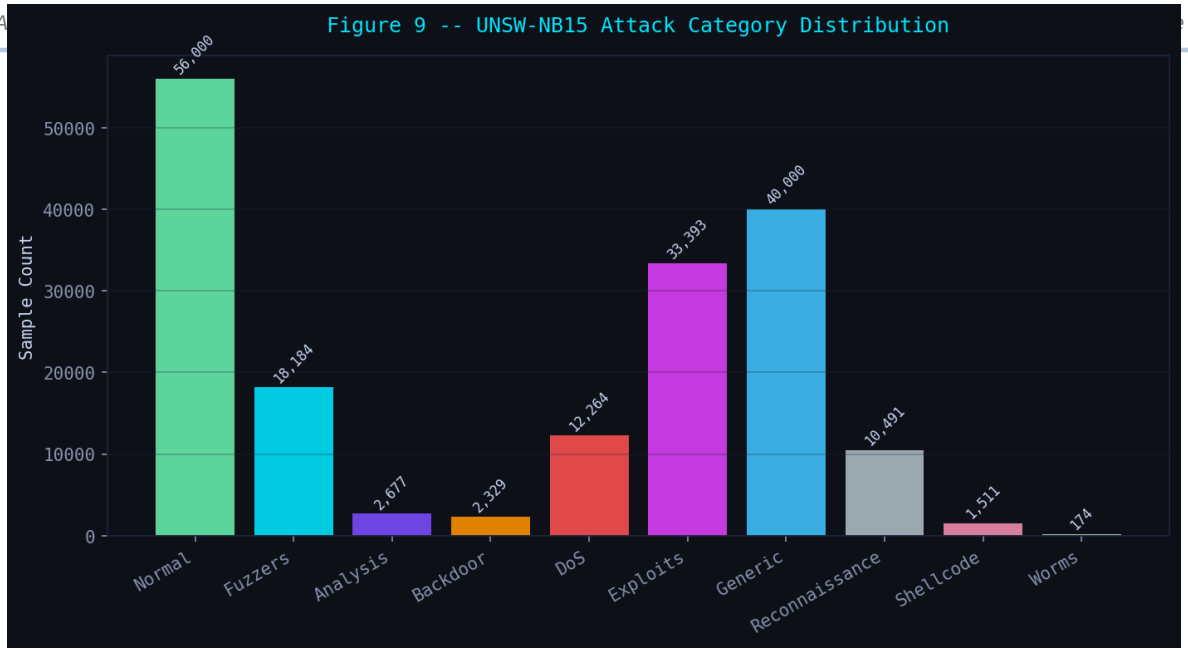

Figure 9 -- UNSW-NB15 attack category

distribution (175, 341 flows).

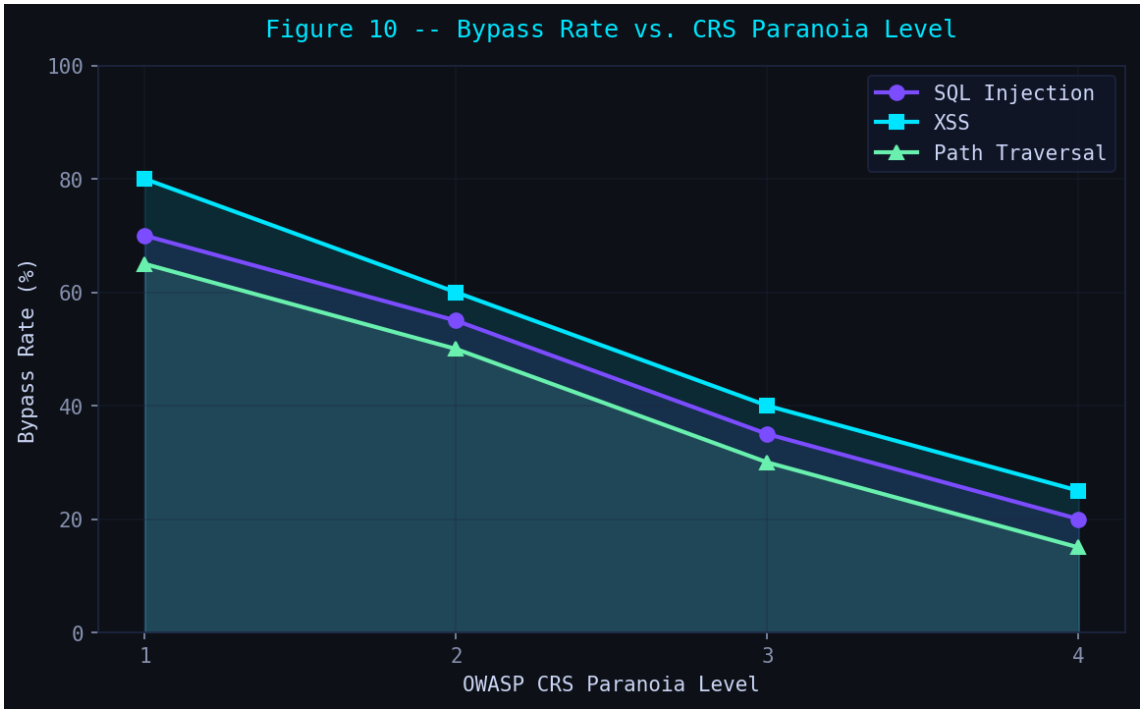

Figure 10 -- Bypass rate vs. CRS Paranoia Level (security vs. usability).